Decide first, execute later

Study date: 20 October 2026. Integrated: 23 October 2026. Task: L002a, hand decision/execution timeline. Sidiq review status: Not yet reviewed.

Research question and hypothesis

What may a decision know before a later execution, and when should the account change? We predict that a decision using an already available mark of 50 leaves cash 800 and four shares unchanged. A later assumed purchase of two shares at 52 with fee 3 leaves cash 693 and six shares. A future fill price cannot be an input to the earlier decision.

This is one intuition lesson following L001f's accounting reproduction, studied October 19 and integrated today. The hand fixture is synthetic, with four ordered teaching events, no market dates and no historical sample. Its metrics are exact row/state/wealth agreement and rejection of three deliberately inconsistent copies. There is no fitted rule or return estimate. The protocol is to write the hand table, verify it independently using Decimal per-share cash movements, replay twice, and retain the failing-copy diagnostics.

Beginner explanation and terms

An observation is information received, such as a price. A decision is a choice based on what is available at that moment. An order is a request; it is not yet an owned position. A fill is an execution that exchanges cash for shares. A mark values shares already owned. These four ideas can concern different prices and occur at different times.

A backtest reads a complete historical file, but its decision must act as though later rows are hidden. Otherwise it has look-ahead bias: access to information it could not have known. An event number is our simple ordering device today. Event 0 is earlier than event 1; these numbers do not claim seconds, exchange sessions or realistic latency.

Our illustrative rule requests two shares if the available mark is at most 50. This rule exists only to make the timeline concrete. We have no evidence that it predicts anything. At event 1 the only price input is 50, received at event 0. A price of 52 will arrive at the assumed fill event 2. The decision cannot consult that later price to change its earlier request.

Hand timeline

All cash, prices and fees use whole arbitrary money units; positions use whole shares.

Event	Meaning	Cash / shares	Mark / wealth
0	Receive observation	800 / 4	50 / 1000
1	Request two shares	800 / 4	50 / 1000
2	Assume full fill at 52; fee 3	693 / 6	52 / 1005
3	Later valuation only	693 / 6	49 / 987

Read the second row slowly: deciding to buy does not already mean owning six shares. We have not modelled reserved buying power; reported cash is the simple accounting balance. The complete-fill assumption is declared, not derived from a quote or historical bar.

Mathematics with units

Let C be cash, q the number of shares, m the mark, n the requested quantity, p the fill price and f the fee. Holdings H = q*m and marked wealth W = C+H. Money plus money is valid; adding shares directly to money is not. At the decision, C = 800 and q = 4 remain unchanged.

At the assumed fill, n*p = 2*52 = 104 money units buys the shares. The cash debit is 104+3 = 107. Therefore C_new = 800-107 = 693 and q_new = 4+2 = 6. Marking the six shares at 52 gives holdings 312 and W_new = 693+312 = 1005.

Why is wealth now five higher despite a fee? Revalue the original four shares before comparing: at mark 52, pre-trade wealth is 800+4*52 = 1008. The buy reduces that same-mark wealth by three, to 1005. Relative to event 0, the old holding gained 4*(52-50) = 8 and the fee cost 3. Thus the net change is 8-3 = 5. Do not call the five a profit created by the act of buying.

At event 3 there is no trade. Cash remains 693 and six shares marked at 49 are worth 294. Wealth is 693+294 = 987. The change from event 2 is 6*(49-52) = -18. Across the complete fixture, wealth changes from 1000 to 987. These invented outcomes are arithmetic, not evidence of expected returns, a strategy backtest or a measured investment opportunity.

The timing rule

For this fixture we require available_event < decision_event < fill_event, namely 0 < 1 < 2. The strict inequality deliberately avoids same-event ties. It is our teaching convention, not a universal exchange rule. Real simultaneous timestamps require an explicit event order, receipt policy and latency assumptions. The next contract lesson will define bounded behavior.

Using 50 as the actual fill price would leave cash 697 rather than 693. That four-unit difference is two shares times a two-unit price difference. The last observed price does not establish what an order will execute at. We assume 52 here solely to illustrate the separation. No spread, queue, volume or market impact model justifies that assumed price.

Implementation boundary

The saved timeline.json is a hand-authored teaching table. verify_timeline.py checks that table with Python Decimal and individual share debits/additions. It is a reference check, not a new Python strategy project or an execution engine. No new C++ syntax is needed for today's intuition. C++ remains the default for the later bounded component after L002b defines its contract.

From the repository root run: `.venv/bin/python scripts/verify_decision_timeline.py`. The verifier checks event ordering, the observation used, the requested quantity, unchanged state before execution, the fill debit, subsequent unchanged cash and every wealth value. It accepts this fixed fixture shape only. It does not implement arbitrary event ingestion.

Checks and observed results

The four rows agree with independent per-share Decimal accounting. Two executions produce identical result JSON, including a SHA-256 fingerprint of the fixture. The decision state is 800 cash and four shares; the post-fill state is 693 cash and six shares; final wealth is 987.

Three copied fixtures are rejected. Giving the decision a price whose availability is event 2 fails the availability condition. Debiting cash and adding shares at event 1 fails state timing. Claiming post-fill cash 697, as though execution used the old price 50, also fails state timing. Original inputs stay intact. These are fixture-integrity controls, not a general proof that a trading engine is invariant to future-price changes; L002d must test that later implementation.

A real Python -O invocation is refused so assertion checks cannot silently disappear. Ruff lint and formatting pass after correcting one long line. No runtime arithmetic check failed. The shared project make check passes 608 Python tests and the existing C++/quality checks, including today's integrated fresh accounting reproduction. No dependency was added.

Costs, failure cases and limitations

The fee is charged only at this assumed full fill. At the decision row, neither fee nor notional is debited. We do not yet specify cancellation, missing fills, insufficient later cash, partial fills or rejection. L001's isolated try_buy rejects invalid or unaffordable requests; that does not automatically define order lifecycle behavior for L002.

The fill and mark happen to equal 52 at event 2. They need not match in reality. Our fee is a fixed teaching number, not a broker tariff. Spread, slippage, taxes, settlement, financing, impact and order priority are not modelled. A received historical price can itself be revised or delayed; real data needs the preserved Q004a/Q005b licence and provenance gates.

The verifier and hand table share an intended specification and can share a flawed assumption. Passing them establishes the stated arithmetic and timing consistency only. No exchange fidelity, market profitability, personal mastery or measured coverage percentage is claimed.

Interview explanation and next step

I separate the information used to decide from the execution that changes the account. I keep the decision state unchanged, use the later assumed fill for the cash debit, and reconcile wealth at a common mark so old-position price changes are not confused with trade costs. I label the fill assumption and test deliberately inconsistent timeline copies.

Next is L002b: define event ordering, fee timing and missing-fill behavior before implementing one small C++ delayed-order component. Future-price invariance, adverse fills and reproduction remain separate L002d-f lessons. C003b, C004 returns/vectors and C005 statistics remain unfinished; trend implementation and later strategy windows stay conditional. Do not compress lessons.

Integration on October 23 preserves the original October 20 study date and results. All 19 original input/artifact records verify; the changed backlog matches its original Git blob. The verifier now reads the published experiment fixture and make decision-timeline joins make check. Two replays and all controls are revalidated. L002b is already complete locally from October 22; the next new lesson is L002c. The weekly synthesis was integrated October 22. Personal review stays Not yet reviewed; lawful access evidence remains required before real data.

Primary sources consulted on October 20

QuantConnect's official time-model documentation explains how LEAN presents past/present data and uses bar end times to control delivery. It motivates our information boundary; this project neither uses LEAN nor inherits its correctness: https://www.quantconnect.com/docs/v2/writing-algorithms/key-concepts/time-modeling/timeslices

The SEC's investor education page explains that a last-traded price need not be the execution price of a market order. Our specific later fill of 52 is an invented assumption: https://www.investor.gov/introduction-investing/investing-basics/how-stock-markets-work/types-orders

Python documents subprocess result codes and captured output, used in today's reproduction and failure-control tooling: https://docs.python.org/3/library/subprocess.html

Clang documents the sanitizer used by the existing buy verifiers in the integrated reproduction: https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

The table, example rule, event inequalities and arithmetic are project derivations. The sources do not endorse the invented rule or establish its returns.

Check your understanding

1. At event 1, what are cash and shares, and can the decision inspect the later fill of 52?

2. Why is post-fill cash 693 rather than 697?

3. Wealth rises from 1000 to 1005 by event 2. Did buying create a five-unit profit?

4. If the event 3 mark becomes 49 with no trade, which account fields change?

5. Does rejecting the three corrupted tables prove a reusable engine has no look-ahead bias?

Suggested answers

1. Cash is 800 and shares are four. The fill is later, so 52 is unavailable to that decision. The request for two shares does not already alter the accounting balance.

2. Two shares cost 104 at the assumed fill of 52, plus fee 3; 800-104-3 = 693. Cash 697 would incorrectly use the earlier observed price of 50 for execution.

3. No. The original four shares rise in marked value by eight and the fee costs three. At a common mark of 52, wealth falls from 1008 immediately before the buy to 1005 after it.

4. Neither cash 693 nor shares six changes. Marked holdings become 294 and wealth becomes 987. The change is a valuation change, not cash received or paid.

5. No. It validates a fixed hand fixture against its declared rules. A later implementation needs independent timing tests and a deliberately leaking control, including future mutations.